Paper.io Agent — Controller Design & Pipeline

Oct 8, 2026 · @Andy

Overview

The controller tracks the hidden opponent with a belief filter, scores candidate action-states with a learned evaluator, and acts through an explicit state machine. This document is the build spec to continue from in Claude Code.

The task. Design and implement a state-machine-based controller for a Paper.io-style game: a 50 × 50 board, 2,000 turns per player, a 3 × 3 starting territory, and an 11 × 11 view. The submission is the controller code plus a concise state-machine design.

What reviewers grade. State-machine design and reasoning, code quality, extensibility, and testability. Game score is feedback, not the target. Every design choice has to be explainable in the interview.

The pipeline in one line. Observation → belief over the hidden opponent → score candidate action-states → the state machine commits and moves.

Design principles

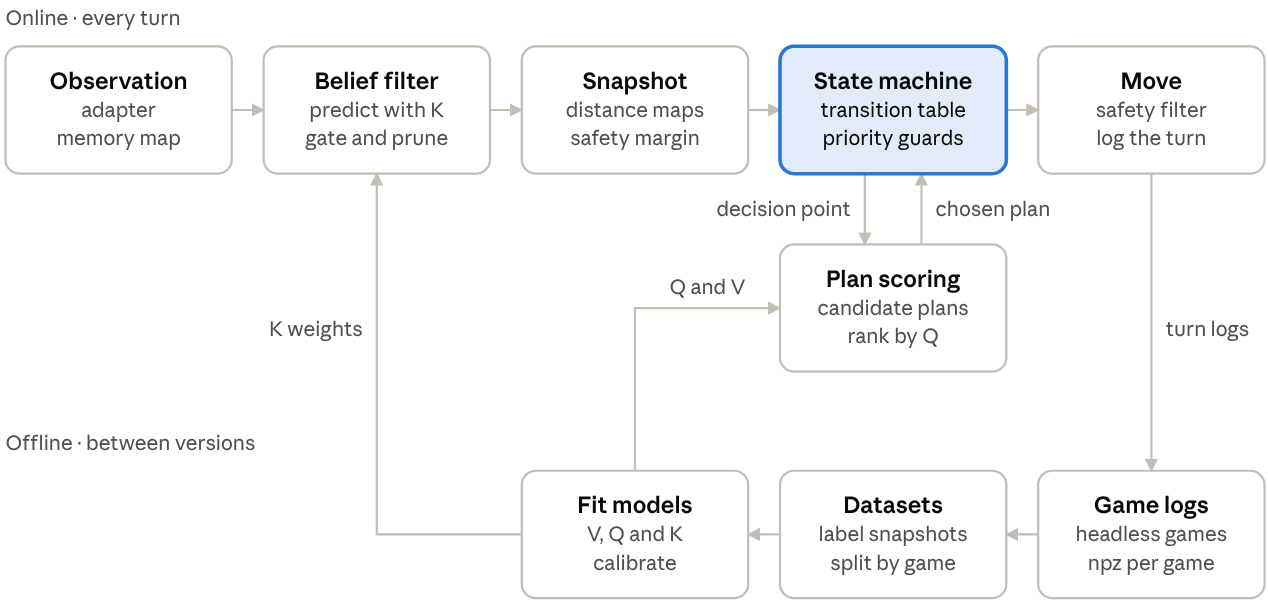
- The state machine is explicit, with a declarative transition table. Learned models feed its guards and plan choice; they never replace transitions.
- Safety is worst-case and hard-coded. A learned score can never override it.
- Expensive geometry (flood fill, reachability) is computed exactly. Models only learn weights on top of it.
- Complexity is added only after a measured failure, and every stage keeps a simpler fallback.

Using this with Claude Code. The game framework already exists; this project adds an agent package and its data structures beside it. Build in the milestone order from Integration and build milestones. Each later section is a self-contained spec for one module. Before coding a module, check its entries in Assumptions to verify. The document assumes the Python starter, so NumPy and PyTorch run in-process.

Architecture at a glance

Two loops make up the project. The online loop runs once per turn inside the controller. The offline loop generates games and refits the models between controller versions.

Each turn flows left to right; offline fitting feeds K, Q and V back in



pipeline · online turn loop and offline fitting loop

Every turn runs left to right along the top row, and at decision points the state machine asks plan scoring for a plan. Logged turns drive the offline row, whose fitted K, Q and V flow back into the belief filter and plan scoring.

Online loop (every turn)

1. Ingest the observation and update the memory map.
2. Update the belief over the opponent: predict with K, gate on evidence, merge, prune.
3. Build the per-turn snapshot: distance maps, safety margin, kill windows.
4. Step the state machine through its transition table.
5. At decision points, generate candidate plans, score them with Q, and commit to the best.
6. The current state picks a move, the safety filter checks it, the move is returned, and the turn is logged.

Offline loop (between versions)

1. Run headless games against an opponent pool and write logs.

2. Assemble labeled datasets for V, Q, and K.
3. Fit the models.
4. Check calibration and win rate, export the parameters, and repeat.

Module map

Module	Responsibility	Spec section
Memory map	Last-seen board contents and timestamps	Belief filter
Belief filter	Weighted hypotheses about the opponent	Belief filter
Motion model K	Probability of each opponent move	Opponent motion model K
Transition engine	Successor states, captures, hashes	Fast batched transitions
Snapshot	Distance maps, safety margin, kill windows	Per-turn snapshot
State machine	States, transition table, safety filter	Controller FSM
Planner	Candidate plans and their features	Plan generation and scoring
Evaluator	V and Q scoring	Plan generation and scoring; Learned models
Recorder	Per-turn logs for training	Offline pipeline

Glossary

Term	Meaning
Hypothesis	One complete guess of the hidden opponent state (position, trail, territory changes), with a weight
Bucket	All hypotheses that share one opponent position
Residual	A bucket's mass whose history was pruned away (history unknown)

Term	Meaning
Support	Every cell with nonzero probability of holding the opponent
Action-state	A state-machine behavior: SAFE, EXPAND, RETURN, HUNT
Plan	An action-state plus its parameters, such as one specific loop path
V(s)	Win probability from state s
Q(s, a)	Expected V at the moment action-state a finishes
K(s, s′)	Score of an opponent successor; a softmax over legal moves turns scores into probabilities
Threat mass	Probability the opponent can cut a planned path before it closes
Safety margin	Earliest opponent arrival on my trail minus my time to reach home

Integration and build milestones

The framework is left untouched. This project adds an agent package and its data structures beside it, and connects through exactly two points: the candidate controller file and one adapter that converts framework types into our own.

- **Framework code is read-only.** Rules and types we need are wrapped, never edited, so upstream changes only touch the adapter.
- **One boundary.** The candidate controller file becomes a thin wrapper that calls agent.Controller. Only adapter.py and offline/selfplay.py import framework internals; selfplay drives the engine headless and reads its true state for logging.
- **Our own types inside.** Everything past the adapter uses the types in Data structures, so each module can be tested without the framework.
- **The engine as oracle.** Reference-transition tests drive the real engine through the same adapter.

What we add (folder names are suggestions; place agent/ wherever the candidate file can import it)

```
<existing framework>/        # unchanged
  <candidate controller file>  # thin wrapper: builds and calls
agent.Controller
agent/
  adapter.py                # framework types -> Observation, MoveResult,
Direction
  controller.py             # Controller: owns the Context, runs the turn
loop
  context.py                # per-match state: memory, belief, models,
recorder
  config.py                 # every tunable constant in one place
  data/                     # data structures (see Data structures)
    world.py                # WorldState: one dense, complete board
    memory.py               # MemoryMap
    hypotheses.py           # HypothesisStore, TerritoryDiffs, ResidualGrid
    snapshot.py             # Snapshot
    plan.py                 # Plan, CandidateBatch
    records.py              # TurnRecord, PlanRecord, GameRecord
  fsm/
    states.py               # SafeState, ExpandState, ReturnState, HuntState
    transitions.py          # transition table + named guard functions
    safety.py               # final move filter
  belief/
    filter.py               # predict, gate, merge, prune, reseed
    motion.py               # K: features, softmax, epsilon floor
    transition.py           # batched successor generation
  geometry/
    bitboard.py             # row-word bitboards, bit tests, flood fill
    distance.py             # BFS by dilation, taxicab transforms
    zobrist.py              # hash tables
  planning/
    candidates.py           # loop, hunt, stay generation
    features.py             # threat mass, closing time, gain
    evaluator.py            # Q scoring, hysteresis
  models/
    value.py
    q_model.py
    k_model.py
offline/
  selfplay.py
  dataset.py
  train_v.py
  train_q.py
```

```
  train_k.py
  evaluate.py
tests/
  reference/                # slow, obviously correct implementations
  test_*.py
```

Milestones

1. **Minimal state machine.** SAFE, EXPAND with small fixed rectangles, RETURN by shortest path home, and the safety filter. Guards use plain distances. It plays a full match against EnemyStateMachine.
2. **Memory and worst-case reachability.** Track the last sighting; the support becomes a diamond that grows one cell per unseen turn. Add the safety-margin guard.
3. **Belief filter v1.** One hypothesis per bucket plus residuals, territory-count evidence, and the hand-feature K.
4. **Plan generation and a hand-tuned evaluator.** Candidate loops, threat mass, and expected utility with a hand-set σ and aggression rate.
5. **HUNT and kill windows.**
6. **Logging and offline fitting.** Fit σ, the aggression rate, and K′s weights, then Q. Add a CNN for Q and the GPU paths only if measurements justify them.

Each milestone ends with passing tests, a match series against EnemyStateMachine, and a one-line note of the failure that motivates the next milestone.

Data structures

All agent state lives in a small set of explicit types, converted from framework types once, at the adapter. Dense types exist for clarity, tests, and logging; compact types exist for speed in the belief filter and planner.

Conventions

- **Cells:** (y, x) with y the row, 0 to 49. The flat index is c = 50·y + x. Flat direction offsets are N −50, E +1, S +50, W −1; always bounds-check x before applying ±1, because flat indices wrap across row ends.
- **Cell codes** in dense boards (int8): 0 free, 1 my territory, 2 my trail, 3 opponent territory, 4 opponent trail, −1 unknown (memory only).
- **Bitboards:** 50 int64 words per board, bit x of word y is cell (y, x). A batch of boards is an (N, 50) array.

- **Backends:** HypothesisStore and CandidateBatch hold torch tensors on the configured device; torch also runs on CPU. Everything else is NumPy.

The types

Type	Holds	Layout	Lifetime
Observation	One turn's input: my position, respawn, trail, turns left, both counts, the visible window and its origin, opponent sighting, last move result	Dataclass of ints and NumPy arrays	One turn
WorldState	One complete board: cell codes, both positions, both trails in order, both counts	int8 (50, 50) plus scalars	Reference transitions, tests, logged ground truth
MemoryMap	My best knowledge of the board	int8 (50, 50) cell codes plus int32 (50, 50) last-seen turn	Whole match
HypothesisStore	All opponent hypotheses	Parallel arrays at fixed capacity plus an active count	Whole match, rewritten each turn
TerritoryDiffs	Out-of-view territory changes, shared by hypotheses	id → (added, removed) bitboards; id 0 = no change	Whole match
ResidualGrid	History-unknown mass per cell	float32 (50, 50)	Whole match
BeliefView	Published belief outputs	Position marginal float32 (50, 50), support bool (50, 50), trail marginal (computed on demand), events	One turn

Type	Holds	Layout	Lifetime
Snapshot	Derived maps and margins	int16 (50, 50) distance maps, scalar margins, kill-window list	One turn
Plan	One committed action-state	Kind, int16 cell path (L,), pointer, expected closing turn, expected gain, score	Until the action-state ends
CandidateBatch	Every candidate at a decision point	Path bitboards (C, 50), capture bitboards (C, 50), features (C, F)	One decision
TurnRecord, PlanRecord, GameRecord	Training logs	Preallocated NumPy buffers, saved as one .npz per game	One game

HypothesisStore fields

Field	dtype	Shape	Meaning
pos	int16	(cap,)	Opponent position, flat index
last_dir	int8	(cap,)	0 to 3, or −1 inside territory
trail_len	int16	(cap,)	Cells in the current trail
trail_bb	int64	(cap, 50)	Trail bitboard
terr_id	int32	(cap,)	TerritoryDiffs id
count	int16	(cap,)	Opponent territory count
turns_out	int16	(cap,)	Turns spent outside territory
flags	uint8	(cap,)	Bit 0 died, bit 1 captured, bit 2 killed me
weight	float32	(cap,)	Probability mass, renormalized every turn
hash	int64	(cap,)	Zobrist hash

Active entries occupy indices 0 to n − 1. Capacity is the maximum hypothesis budget, allocated once at match start.

Core type sketches

```
class MoveResult(Enum):
    NORMAL = 0; CAPTURED = 1; DIED = 2; INVALID = 3

@dataclass
class Observation:
    turn: int
    my_pos: int                    # flat index
    respawn: int
    my_trail: np.ndarray           # (L,) int16 flat cells, oldest first
    turns_left: int
    my_count: int
    opp_count: int
    window: np.ndarray             # (h, w) int8 cell codes, clipped at
edges
    window_origin: tuple[int, int]  # (y, x) of window[0, 0]
    opp_seen_at: int | None        # flat index if visible
    last_result: MoveResult

@dataclass
class WorldState:
    cells: np.ndarray              # (50, 50) int8 cell codes
    my_pos: int
    opp_pos: int
    my_trail: list[int]            # in walking order
    opp_trail: list[int]
    my_count: int
    opp_count: int

@dataclass
class MemoryMap:
    cells: np.ndarray              # (50, 50) int8, -1 = never seen
    last_seen: np.ndarray          # (50, 50) int32 turn number, -1 = never
```

Conversions

- **adapter:** framework observation → Observation, and Direction ↔ the framework's move type.

- **Hypothesis + MemoryMap → WorldState:** for debugging, oracle tests, and building K′s inputs.
- **Engine state → WorldState:** in self-play, for logged ground truth.

Invariants (assert them in debug mode and in tests)

- MemoryMap cells inside the current window always equal the observation.
- Hypothesis weights plus the residual grid sum to 1 after normalization.
- trail_len equals the popcount of trail_bb, and pos is never a bit of its own trail.
- hash equals the Zobrist hash recomputed from the fields.
- A hypothesis expanded over the MemoryMap agrees with every visible cell.

Belief filter

The belief is a weighted set of hypotheses about the opponent, grouped into one bucket per opponent position. Pruning only reshapes what is inside a bucket, so the probability of the opponent being at each cell is never distorted by it.

Representation

- **Base map (shared):** the last-seen contents of every cell, with a timestamp. Only the current window is certain.
- **Hypothesis fields** (parallel arrays, one entry per hypothesis): position, last direction, trail length, trail bitboard (50 int64 words), territory-difference id (0 = no difference), opponent count, turns outside territory, event flags (died, captured Δ, killed me), weight, 64-bit hash.
- **Buckets:** one per cell, holding its hypotheses plus a residual weight for history-unknown mass.
- **Backstop:** the last confirmed sighting (cell, turn).
- **Initialization:** the opponent at its spawn, count 9, empty trail. If the spawn is not visible, split weight across the candidate spawns.

Where each kind of knowledge goes

Knowledge	Goes in	Mechanism
Game rules	Predict step	Impossible successors are never generated
Certain observations	Update step	Likelihood 0 or 1; zero-weight states are deleted

Knowledge	Goes in	Mechanism
Opponent tendencies	Motion model K	Probabilities, never 0 for a legal move

Remembered cells outside the window are a prior, never certain evidence. The opponent may have changed them since they were seen.

Two tiers of certain evidence

- **Position-level** evidence constrains only where the opponent is: a sighting, an empty window, the opponent killing me. It is exact even after pruning, so it applies to whole buckets and to residuals.
- **History-level** evidence depends on the sub-state: count changes and visible cell contents. It applies exactly to hypotheses. A residual is deleted only when the evidence implies a position it cannot satisfy; for example, a capture means the opponent is now on its own territory.

What eliminates what

Evidence this turn	Eliminates	Tier
Opponent visible at cell c	Every state with position ≠ c	Position
Opponent not visible	Every state with position inside the edge-clipped window	Position
I died without stepping on my own trail	Every state whose move did not land on my trail	Position
I did not die	Every successor that stepped on my trail	Position
Invalid move into an in-bounds cell	Every state with the opponent elsewhere	Position
Opponent count dropped to 9	Everything except death and respawn successors	Both
Opponent count rose by Δ	Everything except captures with exactly +Δ	History
Opponent count unchanged	Captures, and deaths if the count was above 9	History

Evidence this turn	Eliminates	Tier
My count dropped by Δ, not explained by my own action	Everything except captures that enclosed exactly those cells	History
A visible cell's contents	Every hypothesis predicting different contents there	History

An opponent at exactly 9 cells who dies stays at 9, so the count cannot reveal that death. Keep those death successors alive and let sightings settle it.

Per-turn update order

1. Apply my last move to every hypothesis: flip my captured cells; if my new cell is on a hypothesis's trail, that opponent dies and respawns.
2. Predict: expand legal moves, resolve their consequences, weight by K, and spread the residuals.
3. Merge duplicates by hash.
4. Gate on position-level evidence.
5. Gate on history-level evidence.
6. Normalize. If total mass is near zero, reseed from the last sighting and log the reseed.
7. Prune each bucket to a budget proportional to its mass (at least 1). Discarded mass goes into that bucket's residual.
8. Publish the position marginal, support, trail marginal, and collapse events.

Ordering rules

- Eliminate before pruning, or the only consistent hypothesis may already be gone.
- Apply my own move before predicting, so predicted counts include my captures.
- Merge before gating, because it is cheaper.
- Normalize only after all gating.

Opponent motion model K

K scores each legal successor of a hypothesis. A softmax over the legal moves, mixed with a small floor ε, turns the scores into move probabilities:

```
P(a \mid s) = (1-\varepsilon)\,\frac{\exp\big(\beta\,K(s, s'_a)\big)}
{\sum_{a' \in \mathrm{legal}} \exp\big(\beta\,K(s, s'_{a'})\big)} +
\frac{\varepsilon}{|\mathrm{legal}|}
```

β sets how sharply the opponent prefers its best move. ε guarantees no legal move ever gets zero probability, which the elimination step depends on.

Successors first. Resolve each move with the rules (death, capture, killed me), then score it. This keeps the rules in one place, handles 2 to 4 legal moves without special cases, gives consequence features for free, and treats all four directions alike. Off-board moves and moves into my cell are dropped, since the harness re-asks after an invalid move.

Default features for K

Feature	Computed from	Intended effect
Steps on own trail	Successor flag	Large negative, but not −∞
Steps on my trail	Successor flag	Large positive
Captures Δ cells	Successor	Positive, scaled with Δ
Change in distance to their territory × trail-length pressure	Shared distance map	Pulls them home as their trail grows
Change in distance to my trail, only if it is inside their 11 × 11 view	Distance map and view check	Aggression
Same direction as last move	Last two trail cells	Momentum
Leaving territory when inside	Successor	Tendency to explore

If K is learned. Show it what the opponent sees: an 11 × 11 crop centred on the opponent, rotated so the candidate move points up, plus the consequence flags. Train it on true states from the logs, because in the filter each hypothesis is treated as a true world. This is the opposite of Q, which trains on beliefs.

Training loss. Softmax cross-entropy over the legal candidates, with the index of the move actually taken as the label:

```
logits = K(views, consequences)             # (B, 4) scores, one per move
logits = logits.masked_fill(~legal, -1e9)   # illegal moves cannot win the
softmax
loss = F.cross_entropy(logits, chosen)      # chosen = index of the actual
move
```

Cost tiers. K runs for every hypothesis and every move, about 80,000 times per turn at a 20,000-hypothesis budget.

1. Feature-based K with fitted weights: microseconds per evaluation. This is the default.
2. Features plus a tiny learned term on the crop, cached by crop contents.
3. A full CNN on the crop, only with a much smaller hypothesis budget.

Evaluation

- One-step held-out log-likelihood of the actual move, against baselines: uniform over legal moves (about −1.1 nats with 3 legal moves) and momentum only.
- Multi-step accuracy: the probability the belief gave the opponent's actual position when it reappears after k unseen turns. If accuracy decays with k, lower β until the k-step predictions are calibrated.

During a match, refit only β and the aggression weight from moves observed while the opponent is visible. Train the base K on an opponent pool, not only EnemyStateMachine.

Possible extension. Add the opponent's hidden mode (expanding or returning) to each hypothesis, with a small table for mode switches. Do this only if multi-step accuracy shows the belief mispredicting turns for home.

Fast batched transitions

Never copy the board. Each hypothesis is a small difference from a shared base map, every candidate move is classified with bit tests, and full successors are built only for the hypotheses that survive gating and pruning.

Bitboards. Store a set of cells as one int64 word per row: 50 bits used, 50 words per board, 400 bytes. Horizontal neighbours are a shift by 1 within a word; vertical neighbours are the adjacent word. Use int64 rather than uint64 because PyTorch's unsigned 64-bit support is limited, and 50 bits never reach the sign bit.

Trail storage	Extend the trail	Is this cell on the trail?
Parent-pointer chain	O(1)	O(trail length)
Bitboard	Copy 400 bytes, set 1 bit	O(1)

Membership is tested for every candidate move, while copying happens only for survivors, so bitboards win. A 20,000-hypothesis budget needs about 8 MB of trail bitboards.

The batched predict step

1. **Expand into (parent, move) pairs** without building anything: new positions are pos[:, None] + DIRS, an (N, 4) tensor, plus a legality mask (in bounds, not my cell).
2. **Classify every pair with bit tests:** on own trail, on my trail, entering own territory. A capture is entering own territory with trail length above 0.
3. **Resolve captures for the capture subset only**, giving Δcount.
4. **Score and normalize:** utilities as an (N, 4) array, illegal moves set to −∞, a softmax per row, mix in ε. Child weight = parent weight × P(move).
5. **Gate on attributes alone:** window, counts, Δcount, my death. A successor's trail is the parent's trail plus one cell, so visible-cell checks need no materialized trail.
6. **Merge by hash, then prune each bucket.**
7. **Materialize the survivors:** copy the parent trail bitboard and set one bit; for a capture, clear the trail and record the territory difference.

```
row, col = cand // 50, cand % 50                    # (N, 4), clamped where
illegal
words = trail_bb.gather(1, row)                     # each parent's row word
on_own_trail = (words >> col) & 1                   # one bit test per pair
```

Captures by bitboard flood fill. A cell is enclosed if it cannot be reached from the board edge without crossing the capturing player's territory or trail. Fill outward from the edge through everything else; whatever the fill misses is captured.

```
def flood(seed, passable):  # (B, 50) int64 arrays: B boards at once
    cur = seed & passable
    while True:
        g = cur | (cur << 1) | (cur >> 1)
        g[:, 1:] |= cur[:, :-1]; g[:, :-1] |= cur[:, 1:]
        g &= passable                          # also clips bit 50
        if (g == cur).all(): return cur
        cur = g
```

- **Inputs:** blocked = territory | trail_with_new_cell, passable = BOARD & ~blocked, and the seed is the passable cells on the board edge.
- **Output:** captured = BOARD & ~flood(seed, passable) & ~territory. Δcount is its popcount; cells of mine inside it give my count drop.
- **Cache** results by (trail hash, territory hash). Different move orders over the same cells produce identical captures.
- On the GPU, check convergence every 8 iterations instead of every one, to avoid stalling on synchronization.

Zobrist hashing for merges. Give every (cell, role) pair a fixed random 64-bit number: position-at-cell, trail-at-cell, territory-gained-at-cell. A state's hash is the XOR of everything true about it, so updates are incremental:

- **Normal move:** h ^= Zpos[old] ^ Zpos[new] ^ Ztrail[new], which is O(1).
- **Capture:** XOR out each trail cell and XOR in each gained cell, which is O(capture size).

Merging is then a hash lookup, or torch.unique on the hash array. At 64 bits, collisions are rare enough to accept.

Verification. Write a slow reference first (copy a full board, apply the move, flood fill with an ordinary search). Check it against the framework engine as an oracle, then check the fast version against the reference on thousands of random states.

Per-turn snapshot

Compute every derived map once per turn, right after the belief update. Every guard and every plan feature reads from the same snapshot, so all decisions in a turn see consistent numbers and each computation is paid for once.

Quantity	How it is computed	Used by
My distance home, from every cell	Breadth-first search by dilation from my territory, with my trail blocked	RETURN, safety margin, closing times
My distance from my position	Breadth-first search from my cell, with my trail blocked	HUNT, candidate generation
Opponent worst-case arrival time	Taxicab distance transform from the support (a conservative lower bound)	Safety margin, threat features
Safety margin	Earliest opponent arrival on any of my trail cells minus my distance home, ±1 for turn order	Priority-2 guard
Kill windows	My distance to likely opponent trail cells versus their distance home	HUNT guard and candidates
Position and trail marginals	From the belief	Features, threat mass

Breadth-first search by dilation. Grow a boolean front one step at a time and record the step at which each cell is first reached. It takes at most about 100 vectorized steps on a 50 × 50 board, the same primitive as the flood fill.

Why a taxicab transform works for the opponent. Only its own trail blocks the opponent's movement, so its true distance is close to the unobstructed taxicab distance and never shorter. That makes the taxicab arrival time a safe, conservative bound. scipy.ndimage.distance_transform_cdt with metric='taxicab' computes it in one call.

Invalid-move re-calls reuse the snapshot. When the harness re-asks after an invalid move, mark the blocked cell and choose again from the cached snapshot without rerunning the pipeline.

Controller FSM

States are thin: each executes a plan and returns one move. A declarative, prioritized transition table owned by the controller decides every state change. Pathfinding, plan generation, and scoring live in shared services that all states call.

State interface

```
class State:
    def enter(self, ctx, plan=None): ...
    def choose_move(self, ctx) -> Direction: ...
    def on_result(self, ctx, result): ...  # advance the plan only on success
    def exit(self, ctx): ...
```

Turn loop

1. Ingest the last result and update the belief.
2. Build the per-turn snapshot.
3. Evaluate transitions in priority order, allowing up to 3 chained transitions per turn, so closing a loop and committing the next plan can happen in one turn.
4. The current state chooses a move.
5. The safety filter checks it: never my own trail, never off the board, never the opponent's cell. On failure, fall back to the move with the shortest safe path home, or else the largest reachable area.
6. Return the direction.
7. choose_move must not advance the plan, because the harness re-asks after an invalid move. Only on_result with a success advances it. After an invalid move, mark that cell as blocked and choose again.

Transition table

Priority	From	To	Guard
0	any	SAFE	I died, so reset at respawn
1	EXPAND, RETURN	HUNT	I reach their trail before they reach mine and before they get home
2	EXPAND, HUNT	RETURN	Worst-case safety margin ⩽ threshold, or the plan became invalid
3	EXPAND, RETURN, HUNT	SAFE	Back on my territory with my trail closed
4	HUNT	RETURN	Kill window closed
5	SAFE	EXPAND	Best plan's score beats staying, by a margin

Each guard is a named pure function over the snapshot, testable on its own. The table doubles as the state-machine diagram for the submission, and adding a state means one class plus its rows. Priority 1 sits above priority 2 on purpose: winning a kill race beats retreating, because killing the opponent also clears the threat to my trail.

The states

State	Owns	Each turn
SAFE	Choosing the next plan	Generate and score candidates. If nothing beats staying, pace inside my territory toward the best exit cell or away from the opponent; there is no stay move.
EXPAND(plan)	Executing one committed loop, including the way back	Check that the next cell is still valid and take it. It never re-chooses its loop; it leaves only through guards.
RETURN	Getting home as fast as possible (abort mode)	Recompute the shortest path to any territory cell, avoiding my trail. Opponent trail cells are passable and stepping on one kills them. With no path home, keep the largest reachable area.
HUNT(target)	Reaching their trail before they close it	Search toward the nearest likely opponent trail cell and retarget as they move. My own safety guard still applies. A confirmed kill goes to SAFE if home, otherwise RETURN.

Movement. Every state follows a list of cells from the planner and replans with a breadth-first search when the path becomes invalid. Blocked cells are my trail, off-board cells, and the opponent's cell. An optional danger cost of $1 + \lambda \cdot P(\text{opponent reaches the cell soon})$ steers paths away from the belief; start with $\lambda = 0$.

Commitment and hysteresis. While exposed, compare continuing with returning each turn, and switch only when the gap exceeds a fixed margin. This prevents dithering between the two.

Plan generation and scoring

At each decision point, generate candidate plans, compute their features from the snapshot and the belief, score each with Q, and commit to the best. The worst-case safety guard stays outside Q, so a mistaken score can never walk the agent into a predictable death.

Candidates

- **EXPAND:** rectangles anchored on the territory boundary, enumerated as exit cell × 4 directions × depth × width, with depth and width capped at about 8 and paths that leave the board filtered out. Loops that run flush along the existing edge gain the most area per trail cell.
- **HUNT:** one candidate per open kill window.
- **STAY:** pace inside the territory for a short horizon.
- **While exposed:** only CONTINUE versus RETURN.

Features

Feature	How it is computed
Planned gain	One batched flood fill over my territory plus each path
Closing time	Travel inside the territory to the exit cell, plus the path length
Threat mass	The sum over cells of P(opponent at c) × [taxicab distance from c to the path < closing time]
Worst-case margin	The minimum over the support of (distance to the path − closing time)
Trail length	Path cells outside my territory
HUNT features	My distance to the target, their distance home, P(the trail cell is real), my own threat mass while hunting
State features	Margin, turns left, both territories, turns since the last sighting, P(opponent is exposed), expected opponent trail length

The taxicab distance from every cell to each candidate path is separable: two sweeps per axis, vectorized across all candidates at once.

What a score means. Q scores an action-state by the expected win probability at the moment it finishes:

```
Q(s, a) = \mathbb{E}\left[\, V(s_{\mathrm{end}}) \mid s, a \,\right], \qquad V(s) \approx \Phi\!\left(\frac{\text{margin}}{s_0 + s_1\sqrt{\text{turns left}}}\right)
```

s_end is the state when the action-state ends: back in SAFE, dead, or at the game's end. Φ is the normal CDF, and the denominator is the uncertainty about the final margin, which shrinks as turns run out.

Why win probability and not cells. Early in the game the curve is nearly linear, so maximizing expected margin is enough. Late in the game it steepens. Ahead by 30 with 400 cells and σ ≈ 15:

Plan	Outcome	Expected margin	Expected V
A	+8 cells, no risk	+8	0.994
B	+40 cells, 5% death	+18	0.95

Expected margin prefers B, but win probability correctly prefers A. When behind, the same curve favors gambles. Always average V over the outcomes; never take V of the average outcome, or the risk sensitivity disappears.

Before Q is trained. Use the decomposed form: P(death) = a × threat mass, with a hand-set aggression rate a and a hand-set σ. Expected V = (1 − P(death)) × V(margin after the capture) + P(death) × V(margin after losing my territory down to 9).

Choosing. Take the argmax, but replace the current plan only if the new best beats it by the hysteresis margin.

Learned models

Three models are trained, each supervised and checked for calibration: V from final results, Q from V at the end of each action-state, and K from observed opponent moves. Each starts small and moves to a bigger model only when held-out games show it underfits.

Model	Inputs	Output	Label	Loss	Fit
V	Margin, turns left, my territory, their territory	P(win) + ½ P(draw)	That game's final result: 1, 0.5, or 0	Binary cross-entropy	L-BFGS
V, margin variant	Same	Final margin as a normal distribution (μ, σ)	Final margin	Gaussian negative log-likelihood	L-BFGS
Q, one model per action type	State features plus action features	Expected V at the end of the action-state	V(s_end)	Cross-entropy with soft labels, or squared error	L-BFGS if linear, Adam if a CNN
K	Transition features, or the rotated crop	One score per legal move	Index of the move actually taken	Softmax cross-entropy	L-BFGS or Adam

Fitting V takes a few lines and no GPU:

```
from scipy.optimize import minimize
from scipy.stats import norm

def loss(p, margin, t, y, w):
    s0, s1 = np.exp(p)                              # keeps both scales
positive
    q = norm.cdf(margin / (s0 + s1 * np.sqrt(t)))
    q = np.clip(q, 1e-9, 1 - 1e-9)                  # avoids log(0)
    return -np.sum(w * (y*np.log(q) + (1-y)*np.log(1-q))) / np.sum(w)

fit = minimize(loss, x0=[0, 1], args=(margin, t, y, w), method='L-BFGS-B')
```

Here w holds per-snapshot weights so that every game counts equally.

Model ladder. Move up a rung only when held-out loss and calibration say so:

1. Linear or logistic regression on the features.
2. The same, plus interaction features. Two worth adding up front: threat mass × my territory, and margin / √(turns left).
3. Gradient-boosted trees.
4. A small CNN with computed channels (the hybrid below).

Q as a hybrid CNN, if it is justified. Stack the board as 50 × 50 channels: my territory, my trail, my position, remembered opponent territory, opponent trail probability, the opponent position belief, staleness, and the board mask. Add computed channels that the network would otherwise have to learn: the plan's path mask, the cells it would capture, my distance home, opponent worst-case arrival, and path threat. Downsample with strided convolutions (50 → 25 → 13 → 7), apply global pooling, concatenate the scalars, and finish with dense layers and a sigmoid. Augment with the 8 rotations and reflections of the board.

Rules that keep the models honest

- Train Q on belief channels, never on the true board. Train K on true states.
- Don't use the Huber loss on margin labels. The outliers it downweights are the deaths, which are the signal.
- Don't rebalance classes. It breaks calibration.
- Turn on exploration while collecting data, or Q never sees what the alternatives do.
- Watch for maximization bias: compare the results of committed plans with their predicted scores.
- Don't discount. The game has a fixed horizon and only the final count matters; V(s_end) labels already give the variance reduction that discounting is usually used for.

Offline pipeline

Generate headless games in parallel, label them, split by game, fit the models, check calibration, and repeat for 2 to 3 rounds until the parameters stop moving.

1. **Generate data** on CPU worker processes, one per core.
    - Play an opponent pool: EnemyStateMachine, BasicStateMachine, a random mover, and older versions of this controller.
    - Turn on exploration: randomize risk settings per game and pick plans with a softmax over their scores.
    - Log the true state from the engine inside the data-generation script, for training K.
    - Run the controllers in the workers on CPU, since many CUDA contexts compete for GPU memory. If Q or K becomes a CNN, route inference through one GPU process that batches requests.
2. **Log** to one .npz file per game:

- Per decision point: V's inputs and the chosen plan's features.
- Per plan: died, killed, cells gained, turns used, and the state when it ended.
- Per opponent move: the true state and the index of the move taken.
- Per game: the final result and final margin, attached to every snapshot.

3. **Assemble datasets** with NumPy. Labels are the final result for V, V(s_end) for Q, and the chosen move for K. Weight each snapshot by 1 / (its game's snapshot count), and split train, validation, and test 70/15/15 by game, or use GroupKFold.
4. **Train.** V and the linear Q and K fit with L-BFGS on CPU. A CNN trains on CUDA with torch.autocast, a pinned-memory DataLoader, batches of 256, Adam at 1e-3, early stopping on validation games, and on-the-fly rotations and reflections on the GPU.
5. **Evaluate.**
   - Calibration: bucket predictions on test games and compare with observed rates.
   - Bootstrap: resample whole games 200 times, refit, and check the spread of the parameters.
   - K: accuracy after k unseen turns, not only one step.
   - A/B: 200 games per variant against EnemyStateMachine. The standard error on a win rate is then about 3.5 percentage points, so smaller differences are noise.
6. **Export and refit.** Save weights as .pt or TorchScript and scalars as JSON. Load them at controller startup, generate new games, and refit.

How much data

Model	Needs	Typical volume
V	A few parameters	A few hundred games
Death-sensitive Q terms	About 10 to 20 events per parameter	About 100 deaths for 5 features, roughly 50 to 100 games
CNN Q or K	Learns geometry from data	Tens of thousands of games

The unit of data is a game, not a snapshot: snapshots from one game share its final label. That is why weighting and splitting are both done by game.

NumPy and CUDA

Use CUDA only for steps that process thousands of items at once. Route everything through one device switch (cuda if available, otherwise cpu) and time each step on both.

Stage	Runs on	Why
Ingest the observation	NumPy	Slice writes into 50 × 50 arrays
Belief predict, gate, merge, prune	GPU at large budgets; NumPy below a few thousand hypotheses	Batched bit tests, torch.unique, bincount
Residual spreading	conv2d with a cross kernel	One convolution per turn
Flood fills for captures and plan gains	GPU, batched	Many boards per call
Snapshot distance maps	NumPy and SciPy	A handful of single 50 × 50 maps
Candidate threat features	GPU, batched over candidates	(C, 50, 50) tensors
CNN inference for Q or K	GPU under torch.inference_mode()	One batched forward pass
State machine, safety filter, move	Plain Python	Tiny
Logging	Preallocated NumPy buffers	No per-turn allocation
Training V and linear Q and K	SciPy L-BFGS on CPU	A handful of parameters
Training CNNs	CUDA	autocast and pinned memory
Self-play workers	CPU processes	GPU contexts compete for memory

Residual spreading as a convolution. The opponent must move every turn, so each residual cell's mass moves to its neighbors and none stays put. That is a convolution with a cross kernel whose centre is 0:

```
X = torch.tensor([[0,1,0],[1,0,1],[0,1,0]], dtype=torch.float32,
device=dev).view(1,1,3,3)
deg = F.conv2d(mask, X, padding=1)               # legal neighbors per cell
spread = F.conv2d(residual / deg.clamp(min=1), X, padding=1) * mask
```

GPU rules

- **A turn needs low latency, not high throughput.** Each GPU operation has a launch overhead of microseconds to tens of microseconds, so small steps are often faster in NumPy.
- **Keep state resident on the device.** Only the observation goes in and only the chosen move comes out.
- **Avoid hidden synchronization.** .item(), .cpu(), bool(tensor), printing, and torch.unique all make the CPU wait for the GPU. Allow a few per turn at most.
- **Preallocate fixed-capacity buffers** at the maximum hypothesis budget, with a count of active entries.
- **Use int64 bitboards**, as described in Fast batched transitions.
- **Keep a working CPU fallback.** Reviewers may run the code without CUDA. Check whether the starter allows NumPy and PyTorch as dependencies at all.

If the project moves to the Java or C++ starter, the pipeline is unchanged. The filter runs natively with plain loops, a CNN runs through ONNX Runtime, and training still happens in PyTorch.

Testing strategy

Every optimized piece is checked against a slow, obviously correct reference, and every model against held-out games. Because most functions are pure, the tests need only small hand-built boards.

Area	Test
Rules oracle	Drive the framework engine with random moves and assert the reference transition reproduces its boards and counts exactly
Fast transitions	Compare the batched transition, flood fill, and hashing against the reference on thousands of random states
Belief: reachability	Opponent unseen for 3 turns from (10, 10): the support is the radius-3 diamond minus my window
Belief: death	A count drop to 9 puts all mass at the spawn

Area	Test
Belief: capture	A count rise of +Δ leaves only paths that produce Δ
Belief: recovery	A contradictory observation triggers a reseed, not a crash
K	A move onto the opponent's own trail gets about ε; with a long trail the move toward home gets most of the mass; on an edge cell the probabilities cover only legal moves and sum to 1
Guards	Each guard returns the expected decision on fixture snapshots
States	Fixture board and context in, choose_move out, asserted
Evaluator	Opponent next to the path gives high threat and one far away gives near zero; the same two plans flip preference between late-and-ahead and late-and-behind
Models	Calibration buckets, bootstrap spread, and multi-step accuracy for K
Regression	Replay recorded matches against EnemyStateMachine and compare decisions
Win rate	A/B series of 200 games per variant

Assumptions to verify and open decisions

These assumptions come from the booklet alone, because the starter's candidate guide could not be loaded while this was written. Check each one before building the module it affects.

Assumption	Affects	How to check
Turns alternate: exactly one opponent move between my calls	The ±1 in every margin, the predict step	Candidate guide, harness loop

Assumption	Affects	How to check
The opponent's spawn is known or mirrors mine	Belief initialization, death collapse	Candidate guide, watching the GUI
How a death caused by the opponent is reported to me	Position-level evidence	Move-result fields
Enclosed means not reachable from the board edge	Capture flood fill	Oracle test against the engine
What happens to an agent inside a captured region	Capture resolution	Oracle test against the engine
Whether there is a per-move time limit	Hypothesis budget, CNN use	Candidate guide
Whether NumPy, PyTorch, and CUDA are allowed	Every GPU path	Candidate guide, or robomstr@uw.edu
Whether I receive my full territory map or only the count	Memory map	Candidate guide
The starter language is Python	NumPy and PyTorch in-process	Confirm before Milestone 1

Open decisions

- [ ] Q: linear per action type, or the hybrid CNN. Decide after the Milestone 6 measurements.
- [ ] K: feature-based, or with a learned crop term.
- [ ] Hypothesis budget per turn, from measured timing.
- [ ] Label for V: the win result, or the final margin with the Gaussian loss.

Interview notes

These are the choices most likely to be probed. For each, be ready with the reasoning and, where possible, the measurement behind it.

Question	Short answer
Why an explicit transition table instead of picking the best-scoring state every turn?	Argmax every turn dithers and hides the reasoning. Guards are named, testable rules, and commitment comes from executing a chosen plan.
Why is safety outside Q?	Death costs all territory. A worst-case reachability guard is certain; a learned score is not.
Why keep the position marginal when pruning, and store discarded mass as a residual?	Top-K only bounds each dropped state at 1% of its bucket, not the total. Renormalizing would make kept hypotheses overconfident, and the residual stops buckets vanishing through pruning artifacts.
Where do rules and certain observations go?	Rules live in the successor function, so impossible states never exist. Certain observations are 0-or-1 likelihoods in the update step.
Why score successors with a softmax?	Rules stay in one place, legality is handled for free, and softmax cross-entropy is the maximum-likelihood loss for which move was chosen.
Why train K on true states but Q on beliefs?	In the filter, each hypothesis is treated as a true world. Q scores my decisions, which must use only what I know.
Why Q(s, a) = E[V(s_end)]?	It folds death and kill outcomes into one label, and averaging V over outcomes keeps the endgame risk sensitivity (Plan A versus Plan B).
Why linear models first?	Few games, rare events, and good features. A CNN needs evidence that linear underfits, plus far more data.
Why no discounting?	The game has a fixed horizon and only the final count matters. V(s_end) labels already reduce variance.

Question	Short answer
Why CUDA only in some places?	A turn needs low latency. GPU launch overhead only pays off on large batches.

The submission itself needs a concise state-machine design: the states, each state's purpose, the important transitions, and the reasoning. Derive it from the transition table and the states table in Controller FSM, plus the scoring summary in Plan generation and scoring.